

Guía técnica de montaje y ejecución del prototipo Deep Gravity Chile

Martín Alonso González Henríquez

2026

1. Propósito y alcance

Este documento describe la estructura, los insumos, las modificaciones y el procedimiento de ejecución del prototipo **Deep Gravity Chile**, implementado para la Provincia de Santiago.

Este directorio constituye la **implementación de referencia** de la rama territorial-relacional del pipeline metodológico propuesto en la tesis. Su propósito no es replicar la corrida documentada—cuyo insumo telco es confidencial y no forma parte de la entrega—, sino poner a disposición el procedimiento completo de transformación, de modo que pueda reejecutarse, examinarse y adaptarse sobre otras fuentes de telefonía móvil de estructura equivalente. La infraestructura territorial es regenerable en su totalidad a partir de fuentes públicas.

El objetivo del prototipo no fue validar un modelo definitivo de movilidad laboral, sino comprobar la factibilidad de adaptar y ejecutar Deep Gravity con datos territoriales y de telefonía móvil disponibles para el caso chileno.

La documentación metodológica completa del proyecto se encuentra en el manuscrito de tesis. Este informe se limita a la información necesaria para utilizar y extender el repositorio.

2. Relación con el repositorio original

El montaje se desarrolló a partir del repositorio oficial de Deep Gravity:

<https://github.com/scikit-mobility/DeepGravity/>

La descarga utilizada como base se realizó el **15 de octubre de 2025**. Se conservaron la arquitectura del modelo, la función de pérdida, el cargador de datos, el procedimiento de entrenamiento y el resto de los archivos originales, incluidos `CITATION.cff`, `osm_query.yaml`, `plots.ipynb` y el directorio `imgs/`, que se entregan sin modificación. El `README` del repositorio original se conserva bajo el nombre `README_original_DeepGravity.md`; el cambio de nombre evita que la plataforma de alojamiento lo presente como documentación de este montaje, y su contenido no fue alterado.

La única modificación de código fuente se realizó en `deepgravity/main.py`. Esta modificación comprende:

- la incorporación explícita de `import torch`;
- el reemplazo de la función `evaluate()` por una evaluación intra-tile explícita y auditable;
- el tratamiento automático de la diagonal según la existencia de autoflujos observados;
- el cálculo separado de `CPC_intra`, `CPC_repo` y de la proporción de flujo observado que permanece dentro del tile;

- la exportación del detalle observado y predicho de cada par origen–destino de prueba en `edges_TEST_pairs.csv`.

Por tanto, la lógica de aprendizaje del modelo original se conserva. La adaptación se concentra en su ejecutabilidad y en la instrumentación de la evaluación del caso chileno.

Componentes originales conservados pero no utilizados

`utils.py` y `data_loader.py` se entregan sin modificación, pero su estatuto difiere. `data_loader.py` se utiliza íntegramente: `FlowDataset` alimenta el entrenamiento y la evaluación. De `utils.py` se utilizan `load_data`, `instantiate_model` y `earth_distance`, mientras que `_compute_support_files` —la rutina que debía generar el contenido de `processed/` a partir de los archivos base— no se emplea. Su funcionalidad fue reconstruida en `processed.ipynb`.

La sustitución no fue una preferencia de implementación. En el repositorio original, la invocación de `_compute_support_files` dentro de `load_data` está comentada, y su guardia `_is_support_files_computed` exige un archivo (`msoa_df_all.csv.zip`) que la propia rutina nunca produce, de modo que nunca podría activarse. La rutina presenta además defectos que impiden su uso: serializa los centroides en `od2flow.pkl` en lugar del diccionario de flujos; almacena cada centroide como la cadena `'[x,y]'` en orden longitud–latitud, mientras `earth_distance` espera un par numérico en orden latitud–longitud; y escribe `tileid2oa2handmade_features.json` en el directorio de trabajo y no en `processed/`, que es donde `load_data` lo busca.

Se documenta aquí porque `utils.py` se redistribuye sin alterar y esa rutina permanece visible en el archivo: la entrega no la modificó, la sustituyó. A efectos de la licencia CC BY 4.0, los cambios declarados siguen siendo únicamente los de `main.py`.

Atribución y estatuto del código heredado

El repositorio original no incorpora un archivo de licencia en su árbol de GitHub. Sus autores, no obstante, depositaron el código en Zenodo bajo licencia **Creative Commons Attribution 4.0 International** (CC BY 4.0):

F. Simini, G. Barlacchi, M. Luca, L. Pappalardo, *Deep Gravity*. Zenodo (2021). <https://doi.org/10.5281/zenodo.5573573>

Ese depósito es, además, el que el **README** del repositorio original solicita citar junto con el artículo asociado:

F. Simini, G. Barlacchi, M. Luca, L. Pappalardo, *A Deep Gravity model for mobility flows generation*, Nature Communications 12, 6576 (2021). <https://doi.org/10.1038/s41467-021-26752-4>

El código heredado que esta entrega redistribuye se rige, por tanto, por CC BY 4.0 (<https://creativecommons.org/licenses/by/4.0/>), que autoriza su redistribución y adaptación bajo tres condiciones: atribuir a los autores, enlazar la licencia e indicar los cambios realizados. Esta entrega las cumple de la siguiente forma:

- **Atribución:** se conserva `CITATION.cff` sin modificación, se mantiene el **README** original bajo el nombre indicado, y tanto el artículo como el depósito de software se citan en el manuscrito de tesis y en esta sección.
- **Enlace a la licencia:** se declara en el párrafo anterior y en el **README** de la raíz del repositorio.
- **Indicación de cambios:** la única modificación de código fuente se detalla en la lista precedente de esta misma sección. Los archivos añadidos por esta tesis son identificables como tales y no se presentan como parte del repositorio original.

Los datos de ejemplo incluidos en el repositorio original forman parte del mismo depósito y se conservan sin modificación.

Los componentes desarrollados en el marco de esta tesis —los notebooks de `preprocessing/`, `config.py`, los productos de `data/chile/` y este informe— se distribuyen bajo **licencia MIT**, declarada en el archivo `LICENSE` de la raíz del repositorio. Dicha licencia no alcanza a los archivos provenientes del repositorio original, que permanecen bajo los términos de CC BY 4.0 señalados arriba.

3. Estructura de la entrega

Ubicación de este directorio

El repositorio `pipeline-telco-chile-aplicaciones` contiene dos directorios de primer nivel; este prototipo constituye uno de ellos. En la raíz de dicho repositorio se encuentran el `README` general, que describe ambos prototipos y el criterio de publicación común, y el archivo `LICENSE`, que declara los términos aplicables a los componentes desarrollados en el marco de esta tesis. El segundo directorio contiene el prototipo de la rama conductual-individualizada.

Árbol de contenidos

La estructura del directorio del prototipo es la siguiente.

```
01-prototipo-deepgravity-chile
|-- README_original_DeepGravity.md    (original, renombrado)
|-- CITATION.cff                      (original)
|-- osm_query.yaml                    (original)
|-- plots.ipynb                       (original)
|-- imgs/                             (original)
|-- requirements_venv_dg.txt
|-- requirements_venv_dg_markdowns.txt
|-- requirements_POIs.txt
|-- Informe_Montaje_Deep_Gravity_Chile.pdf
'-- deepgravity/
    |-- __init__.py                    (original)
    |-- main.py                        (modificado)
    |-- data_loader.py                 (original)
    |-- utils.py                       (original)
    |-- models/                       (original)
    |-- results/                       (vacío)
    |-- preprocessing/
    |   |-- config.py
    |   |-- preprocesamiento_trazas_telco.ipynb
    |   |-- preprocesamiento_POIs.ipynb
    |   '--- processed.ipynb
    '--- data/
        |-- new_york/                  (original)
        '--- chile/
            |-- output_areas.*
            |-- tessellation.*
            |-- features.csv
            '--- processed/
                |-- oa_gdf.csv.gz
                |-- oa2centroid.pkl
                |-- oa2features.pkl
                |-- oa2features_columns.json
                |-- tileid2oa2handmade_features.json
                |-- train_tiles.csv
```

Criterio de publicación

La entrega aplica un criterio único: **se publica el procedimiento completo; no se publican los artefactos ni las salidas derivadas de la fuente de telefonía móvil**. Ello obedece a las restricciones de uso legítimo que el manuscrito registra para los productos de la rama conductual y relacional, y al acuerdo de confidencialidad bajo el cual se accedió a la fuente.

En consecuencia, no forman parte de esta entrega:

- `flows.csv`, producto relacional derivado de las trazas;
- `flows_oa.csv.zip` y `od2flow.pkl`, sus versiones serializadas en `processed/`;
- el contenido de `deepgravity/results/`, que reúne el checkpoint del modelo y las salidas de la corrida.

Todo el código que produce esos objetos sí se publica, de modo que el procedimiento resulta examinable y reejecutable sobre una fuente equivalente.

Por el mismo criterio, los tres notebooks de `preprocessing/` se entregan **sin resultados de ejecución**. Lo que el repositorio ofrece es el procedimiento; los valores de la corrida documentada figuran en la Sección 12 de este informe y en el manuscrito de tesis.

Los términos bajo los cuales se distribuye lo publicado se declaran en la Sección 2 y en el README de la raíz del repositorio: los componentes desarrollados en el marco de esta tesis se rigen por licencia MIT, y los archivos heredados del repositorio original por CC BY 4.0.

4. Disponibilidad de datos y niveles de ejecución

El repositorio distingue dos niveles de ejecución, definidos por el tipo de fuente que requieren y no por el grado de acceso de quien lo utiliza.

Nivel 1 — Infraestructura territorial: reproducible sin restricciones

Toda la cadena que produce `output_areas.*`, `tessellation.*`, `features.csv` y los objetos serializados de `processed/` puede regenerarse a partir de fuentes públicas:

- el extracto nacional `chile.osm.pbf` de OpenStreetMap;
- la cartografía DPA 2023;
- los resultados de población del Censo 2024.

Este nivel no requiere autorización alguna. Los productos correspondientes se incluyen además en la entrega, de modo que pueden utilizarse directamente sin necesidad de regenerarlos.

Nivel 2 — Procedimiento telco: reejecutable sobre fuentes equivalentes

Los notebooks `preprocesamiento_trazas_telco.ipynb` y la sección correspondiente de `processed.ipynb` implementan íntegramente el tratamiento de la fuente de telefonía móvil. Operan sobre cualquier fuente que contenga identificador seudonimizado, marca temporal y coordenadas, mediante el mapeo de esquema declarado en `config.py`. Los códigos administrativos son opcionales, dado que la asignación territorial se resuelve mediante la DPA 2023.

El pipeline no exige un esquema de origen particular: exige que la fuente sea transformable a la representación canónica del proyecto. La fuente utilizada en la corrida documentada

es confidencial y no se distribuye; su ausencia no impide ejecutar el procedimiento sobre otro insumo de estructura comparable.

Consecuentemente, los valores de referencia de la Sección 12 no son verificables por terceros. Lo que el repositorio entrega es el procedimiento, no la reproducción de un resultado particular.

Configuración de rutas

Las rutas y parámetros se centralizan en `deepgravity/preprocessing/config.py`. Las fuentes externas se ubican fuera del árbol versionado, en un directorio que el usuario crea en su propio entorno y cuya ubicación puede definirse mediante la variable de entorno `DGCL_DATA_DIR` o editando el valor por defecto del archivo. Dicho directorio no forma parte del repositorio.

La función `cfg.describe_config()` entrega un resumen de la configuración activa y de la disponibilidad de cada fuente, y `cfg.require()` emite un mensaje explícito cuando alguna no se encuentra, distinguiendo si corresponde a una fuente pública descargable o a una fuente provista por el usuario.

5. Resumen del pipeline de construcción

5.1. Preprocesamiento de trazas telco

En la corrida documentada, el archivo inicial de la Provincia de Santiago contenía **12.020.523 registros** correspondientes a noviembre de 2023.

El procedimiento aplicado comprende:

1. eliminación de duplicados exactos;
2. exclusión de identificadores diagnosticados como anómalos por volumen y estructura incompatibles con movilidad humana ordinaria;
3. tratamiento de registros con el mismo par usuario–timestamp:
 - pares separados por hasta 300 m: fusión mediante promedio aritmético de coordenadas;
 - pares separados por más de 300 m: eliminación;
 - grupos con más de dos registros: eliminación;
4. restricción horaria a 07:00–23:00;
5. asignación comunal mediante la DPA 2023;
6. aplicación de un procedimiento *snap-in-place* de hasta 200 m para puntos sin comuna, sin casos modificados en el subconjunto utilizado;
7. eliminación de usuarios con una sola observación;
8. ordenamiento temporal de los registros de cada usuario;
9. conteo de cambios consecutivos entre comunas;
10. eliminación de transiciones con origen igual a destino;
11. agregación por par dirigido de comunas.

Los umbrales de 300 m y 200 m, la ventana horaria y la evidencia basal mínima son parámetros declarados en `config.py` y ajustables ante fuentes con distinta precisión locacional. La regla de exclusión de identificadores anómalos pertenece al procedimiento; los valores concretos dependen de cada fuente y deben justificarse a partir de su distribución empírica de registros por usuario.

El producto resultante, `flows.csv`, contuvo 992 pares dirigidos:

$$32 \times 31 = 992.$$

Cada valor `flow` representa el número de veces que una observación en una comuna fue seguida por una observación del mismo usuario en otra comuna. Este archivo deriva de la fuente telco y no forma parte de la entrega; se regenera ejecutando el notebook correspondiente.

Advertencia semántica. Estos valores corresponden a **transiciones consecutivas entre comunas**. No representan viajes únicos, residencia observada, lugar de trabajo observado ni una matriz validada de *commuting*. Los nombres `residence`, `workplace` y `commuters` utilizados posteriormente se conservan únicamente por compatibilidad con el repositorio original.

5.2. Atributos territoriales

Las *Output Areas* corresponden a las **32 comunas de la Provincia de Santiago**. Para cada comuna se construyeron 18 variables derivadas de OpenStreetMap:

- 5 variables de uso de suelo, expresadas como fracción de superficie;
- 3 variables de red vial, expresadas como kilómetros de vía por km²;
- 5 variables de puntos de interés, expresadas como conteos por km²;
- 5 variables de edificaciones, expresadas como conteos por km².

Las cinco familias empleadas para puntos de interés y edificaciones son transporte, alimentación, salud, educación y comercio.

Las 18 variables se almacenan en `oa2features.pkl`. Su orden queda registrado en `oa2features_columns.json`.

El vector que el modelo utiliza durante la ejecución no está compuesto únicamente por esas 18 variables: `main.py` antepone a cada unidad un elemento adicional, $\log(O_i^{\text{global}})$. Conviene precisar qué es esa cantidad. `utils.load_data` construye `oa2pop` agregando `flows_oa.csv.zip` por origen, de modo que corresponde al **outflow total observado** de cada unidad y no a su población censal. Se trata de comportamiento heredado del repositorio original, donde los flujos son de *commuting* y esa suma opera como proxy de población trabajadora residente; en este montaje, donde los flujos son transiciones consecutivas entre comunas, la cantidad aproxima la intensidad observacional de la comuna. El vector por unidad tiene, por tanto, 19 elementos, y `dim_input = 39` al concatenar origen, destino y distancia.

La población del Censo 2024 se incorpora a `features.csv`, pero el Chunk 4 de `processed.ipynb` la excluye explícitamente de `oa2features.pkl`: no forma parte de las variables que recibe el modelo. Su uso en este prototipo se limita a las secciones de análisis de `processed.ipynb`, donde alimenta el examen de correlaciones entre atributos comunales y rendimiento por origen.

La delimitación a la Provincia de Santiago acota el ámbito territorial del prototipo y no constituye una restricción del procedimiento, que opera sobre cualquier conjunto de unidades administrativas.

5.3. Teselación y partición espacial

Se utilizó una grilla regular de **25 km**, valor determinado mediante el barrido documentado en el notebook territorial y declarado en `config.TILE_SIZE_KM`. La grilla completa contiene 10 tiles geométricos, de los cuales cuatro contienen puntos representativos de comunas.

La distribución final es:

Tile	Número de comunas	Partición
T002_001	15	Test
T001_001	8	Train
T002_002	6	Train
T001_002	3	Train

La partición contiene 17 comunas de entrenamiento y 15 comunas de prueba. Es determinística y corresponde a una única configuración espacial.

6. Archivos de processed/

El directorio `data/chile/processed/` contiene nueve archivos, de los cuales `main.py` y `utils.load_data` consumen ocho; `oa2features_columns.json` no es leído por el código y cumple una función de procedencia, al registrar el orden exacto de las variables serializadas. Siete se incluyen en la entrega; los dos restantes derivan de la fuente telco y se generan ejecutando el notebook correspondiente.

Archivo	Contenido	Entrega
<code>oa2centroid.pkl</code>	Diccionario <code>OA_ID → [latitud, longitud]</code> . Se utilizan puntos representativos interiores de las comunas.	Incluido
<code>oa_gdf.csv.gz</code>	Tabla mínima de identificadores, con las columnas <code>OA_ID</code> y <code>geo_code</code> .	Incluido
<code>tileid2oa2handmade_features.json</code>	Mapeo entre cada <code>tile_id</code> y las comunas asignadas a dicho tile.	Incluido
<code>oa2features.pkl</code>	Diccionario de las 18 variables OSM normalizadas por comuna.	Incluido
<code>oa2features_columns.json</code>	Orden exacto de las 18 variables almacenadas en <code>oa2features.pkl</code> .	Incluido
<code>train_tiles.csv</code>	Tiles incluidos en el conjunto de entrenamiento.	Incluido
<code>test_tiles.csv</code>	Tile incluido en el conjunto de prueba.	Incluido
<code>flows_oa.csv.zip</code>	Transiciones comunales en el esquema requerido por el repositorio: <code>residence, workplace, commuters</code> .	Generado por el usuario
<code>od2flow.pkl</code>	Diccionario (<code>origin, destination</code>) $\rightarrow$ <code>flow</code> .	Generado por el usuario

7. Entornos computacionales

La entrega incluye tres archivos de dependencias en la raíz del directorio del prototipo.

- `requirements_venv_dg.txt`: entorno utilizado para ejecutar la corrida de Deep Gravity.
- `requirements_venv_dg_markdowns.txt`: entorno utilizado para ejecutar `preprocesamiento_trazas_telco.ipynb` y `processed.ipynb`.
- `requirements_POIs.txt`: dependencias de `preprocesamiento_POIs.ipynb`.

El notebook `preprocesamiento_POIs.ipynb` fue ejecutado en Google Colab, por trabajar exclusivamente con fuentes públicas. Sus dependencias se documentan en `requirements_POIs.txt`, que permite reconstruir un entorno equivalente en local.

Los tres notebooks importan `config.py` desde el directorio `preprocessing/`, por lo que deben ejecutarse teniendo dicho directorio como directorio de trabajo.

Creación del entorno del modelo en Windows

Desde la raíz del directorio del prototipo:

```
python -m venv .venv_dg
.\.venv_dg\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements_venv_dg.txt
```

Creación del entorno del modelo en Linux o macOS

```
python3 -m venv .venv_dg
source .venv_dg/bin/activate
python -m pip install --upgrade pip
pip install -r requirements_venv_dg.txt
```

Para ejecutar los notebooks locales puede repetirse el procedimiento utilizando `requirements_venv_dg_markdowns.txt` y un entorno independiente. Lo mismo aplica a `requirements_POIs.txt` para el cuaderno territorial, cuyo entorno conviene mantener separado por tratarse de dependencias geoespaciales distintas.

8. Orden recomendado de ejecución

Para regenerar el montaje completo desde los insumos correspondientes, el orden recomendado es:

1. ejecutar `preprocesamiento_POIs.ipynb`, que genera la infraestructura territorial, incluido `output_areas.geojson`;
2. ejecutar `preprocesamiento_trazas_telco.ipynb` sobre la fuente telco disponible, previa configuración de `COLUMN_MAP` en `config.py`; este cuaderno consume `output_areas.geojson` para construir las transiciones entre comunas;
3. ejecutar los chunks iniciales de `processed.ipynb`, encargados de generar `data/chile/processed/`;
4. ejecutar `main.py`;
5. continuar con las secciones de análisis y visualización de `processed.ipynb`, las cuales utilizan los resultados generados por el modelo.

Los pasos 1 y 3 corresponden al Nivel 1 y pueden ejecutarse con lo que el repositorio distribuye. Los pasos 2, 4 y 5 requieren una fuente telco propia: sin ella, los objetos `flows_oa.csv.zip` y `od2flow.pkl` no existen y `main.py` no dispone de variable objetivo.

9. Ejecución de la corrida de referencia

El comando debe ejecutarse desde el directorio:

```
01-prototipo-deepgravity-chile/deepgravity/
```

PowerShell

```
$DS="chile"
$OAID="OA_ID"
$FO="residence"
$FD="workplace"
```

```

$FF="commuters"

python .\main.py '
--dataset $DS '
--oa-id-column $OAID '
--flow-origin-column $FO '
--flow-destination-column $FD '
--flow-flows-column $FF '
--epochs 500 '
--batch_size 1 '
--test-batch-size 1 '
--lr 5e-6 '
--momentum 0.9 '
--device cpu '
--mode train '
--seed 42

```

Bash

```

DS="chile"
OAID="OA_ID"
FO="residence"
FD="workplace"
FF="commuters"

python main.py \
--dataset "$DS" \
--oa-id-column "$OAID" \
--flow-origin-column "$FO" \
--flow-destination-column "$FD" \
--flow-flows-column "$FF" \
--epochs 500 \
--batch_size 1 \
--test-batch-size 1 \
--lr 5e-6 \
--momentum 0.9 \
--device cpu \
--mode train \
--seed 42

```

La configuración efectiva de la corrida documentada es:

Elemento	Configuración
Modelo	Deep Gravity (DG)
Épocas	500
Optimizador	RMSprop
Tasa de aprendizaje	5×10^{-6}
Momentum	0,9
Batch de entrenamiento	1 origen
Batch de prueba	1 origen
Dispositivo	CPU
Semilla inicial	42
Early stopping	No
Scheduler	No
Selección del mejor epoch	No
Weight decay explícito	No

La ejecución documentada tardó aproximadamente **84,19 segundos**. El checkpoint guardado corresponde al estado del modelo después del epoch 500 y no forma parte de la entrega.

10. Evaluación implementada

La función `evaluate()` modificada evalúa cada comuna de prueba sobre los destinos pertenecientes a su mismo tile.

Para cada origen i :

1. se identifican los destinos candidatos del tile;
2. se excluye i del conjunto de destinos cuando no existe flujo diagonal observado;
3. se generan las variables de cada par (i, j) ;
4. el modelo produce un logit para cada destino;
5. se aplica un softmax sobre los destinos candidatos;
6. las probabilidades se convierten en flujos mediante:

$$\hat{y}_{ij} = p_{ij} O_i^{\text{intra}},$$

donde:

$$O_i^{\text{intra}} = \sum_{j \in D_i} y_{ij}.$$

Por construcción:

$$\sum_{j \in D_i} \hat{y}_{ij} = O_i^{\text{intra}}.$$

La igualdad entre la masa observada y predicha intra-tile es, por tanto, una restricción del procedimiento de evaluación. El modelo determina la distribución entre destinos, no el volumen total intra-tile del origen.

Se reportan dos versiones del CPC:

$$CPC_{\text{intra}} = \frac{2 \sum_{i,j} \min(y_{ij}, \hat{y}_{ij})}{\sum_{i,j} y_{ij} + \sum_{i,j} \hat{y}_{ij}},$$

y una versión con el denominador global utilizado por el repositorio:

$$CPC_{\text{repo}} = \frac{2 \sum_{i,j \in \text{intra}} \min(y_{ij}, \hat{y}_{ij})}{2 \sum_i O_i^{\text{global}}}.$$

También se calcula:

$$\text{Cobertura}_{\text{intra}} = \frac{\sum_i O_i^{\text{intra}}}{\sum_i O_i^{\text{global}}}.$$

11. Archivos que produce la ejecución

Los archivos descritos en esta sección no forman parte de la entrega: se generan al ejecutar el procedimiento sobre una fuente propia, y se documentan aquí para que su contenido y su función resulten identificables.

Outputs directos de main.py

La corrida del modelo genera, en `deepgravity/results/`:

- `model_DG_chile.pt`: checkpoint final del modelo y estado del optimizador;
- `edges_TEST_pairs.csv`: detalle observado y predicho de los pares intra-tile de prueba;
- `tile2cpc_DG_chile.csv`: métricas agregadas del tile de prueba.

`edges_TEST_pairs.csv` contiene las columnas `origin`, `destination`, `prob`, `y_obs` e `y_pred`.

Productos derivados en processed.ipynb

A partir de los outputs anteriores, las secciones de análisis del notebook generan:

- `matrix_obs.csv`, `matrix_pred.csv`, `matrix_err.csv`;
- `table_top_errors.csv`, `top_over.csv`, `top_under.csv`;
- `origin_performance_metrics.csv`;
- `recall_at_k_by_origin.csv`;
- `bias_vs_distance.csv`;
- `feature_correlations_CPC_i.csv` y `feature_correlations_MAPE_i.csv`.

Los tres outputs de `main.py` son productos directos del modelo. Los once restantes corresponden a productos de auditoría y diagnóstico construidos posteriormente. Las figuras asociadas se escriben en `imgs/`.

12. Resultados de referencia

Los valores siguientes corresponden a la corrida documentada en el proyecto y se conservan como **registro documental**. Dado que dependen de la fuente telco confidencial, no constituyen un control verificable por terceros ni deben esperarse al ejecutar el procedimiento sobre otra fuente.

Indicador	Valor
Comunas de prueba	15
Destinos evaluados por origen	14
Pares dirigidos evaluados	210
Flujo observado intra-tile	354.605
Flujo predicho intra-tile	354.605
CPC_{intra}	0,4594
CPC_{repo}	0,2070
Cobertura intra-tile	0,4505
R^2 lineal	-0,8726
R^2 sobre $\log(1 + y)$	-0,5192

La desviación estándar del CPC por tile aparece como `NaN` porque la partición contiene un solo tile de prueba.

Estos resultados no deben interpretarse como una validación general de Deep Gravity ni como evidencia concluyente sobre movilidad laboral en Santiago. Su función es documentar las condiciones y las magnitudes de la corrida efectivamente realizada.

13. Controles de verificación

Insumos territoriales (Nivel 1)

Estos controles pueden aplicarse sobre lo que el repositorio distribuye:

- 32 valores únicos de `OA_ID`;
- 18 variables en `oa2features.pkl`;
- concordancia entre el orden de las variables y `oa2features_columns.json`;
- cuatro tiles ocupados;
- 17 comunas en entrenamiento y 15 en prueba;
- ausencia de valores `NaN` o infinitos en los insumos;
- concordancia de identificadores entre geometrías y atributos.

Insumos derivados de la fuente telco (Nivel 2)

Estos controles requieren haber ejecutado el procedimiento sobre una fuente propia:

- número de pares dirigidos consistente con el número de unidades territoriales;
- ausencia de diagonal en la matriz de flujos;
- concordancia de identificadores entre flujos, geometrías y atributos;
- existencia de `flows_oa.csv.zip` y `od2flow.pkl` en `processed/`.

Estructura de las salidas del modelo

Tras ejecutar `main.py`, corresponde comprobar que existan los tres outputs directos, que `edges_TEST_pairs.csv` contenga una fila por par dirigido evaluado, que la masa observada y predicha intra-tile coincidan por construcción y que `tile2cpc_DG_chile.csv` contenga una fila por tile de prueba. Se trata de controles estructurales del procedimiento, independientes de los valores de la corrida documentada.

14. Limitaciones de interpretación

El prototipo debe utilizarse considerando las siguientes restricciones:

- los flujos representan transiciones consecutivas entre comunas, no viajes únicos ni commuting observado;
- un mismo usuario puede contribuir más de una vez a un par;
- no se infirieron residencia ni lugar de trabajo;
- no se aplicaron filtros de días hábiles, permanencias o intervalo máximo entre observaciones;
- la escala comunal elimina movilidad intracomunal y oculta heterogeneidad territorial interna;
- la evaluación utiliza una única partición espacial y un solo tile de prueba;
- no se realizaron repeticiones con múltiples semillas;
- no se implementó selección del mejor epoch, early stopping ni scheduler;
- no se dispone de ground truth individual ni de una matriz externa semánticamente equivalente;

- el vector de entrada de cada unidad incluye $\log(O_i^{\text{global}})$, cantidad derivada de la propia matriz de flujos, de modo que la dependencia respecto de la masa observada opera también durante el entrenamiento;
- el modelo recibe además como conocida la masa observada intra-tile de cada origen durante la evaluación.

En consecuencia, el montaje demuestra compatibilidad computacional y permite una evaluación exploratoria de la distribución de destinos. No constituye un modelo validado de movilidad laboral para Chile.

15. Posibles extensiones

El repositorio puede utilizarse como base para:

- evaluar unidades territoriales más pequeñas que la comuna;
- recalibrar el tamaño y la posición de la teselación;
- construir flujos con una semántica de movilidad más específica;
- incorporar permanencias, anclas espaciales o ventanas temporales;
- ejecutar validación espacial repetida;
- comparar múltiples semillas e hiperparámetros;
- incorporar fuentes externas de contraste;
- evaluar otras definiciones de outflow y universos de destinos;
- aplicar el procedimiento a otros territorios, operadores o períodos.

Cualquier extensión debe revisar simultáneamente la definición del flujo, la escala territorial, el conjunto de destinos y el esquema de evaluación.

16. Problemas frecuentes

Problema	Revisión recomendada
No se encuentra el dataset	Ejecutar <code>main.py</code> desde <code>deepgravity/</code> y verificar la existencia de <code>data/chile/</code> .
Faltan archivos procesados	Comprobar que los siete archivos incluidos existan en <code>data/chile/processed/</code> . <code>flows_oa.csv.zip</code> y <code>od2flow.pkl</code> no forman parte de la entrega y se generan ejecutando el procedimiento sobre una fuente propia.
No se encuentra una fuente externa	El mensaje de <code>cfg.require()</code> indica la ruta esperada y si corresponde a una fuente pública descargable o a una fuente provista por el usuario. Ajustar <code>config.py</code> o la variable <code>DGCL_DATA_DIR</code> .
La fuente telco no se transforma al esquema canónico	Completar <code>COLUMN_MAP</code> en <code>config.py</code> con los nombres de campo de la fuente utilizada. El mensaje de error indica los campos ausentes y las columnas disponibles.
Error con identificadores	Verificar que los códigos comunales se carguen como texto y coincidan entre todas las fuentes.
Error al cargar el checkpoint	Confirmar que el entorno y la arquitectura coincidan con los utilizados durante el entrenamiento.
<code>stdev = NaN</code>	Es el comportamiento esperado al disponer de un único tile de prueba.
Resultados diferentes a los de referencia	Es lo esperable al ejecutar sobre otra fuente. Los valores de la Sección 12 corresponden a una corrida específica y no constituyen un control replicable.
<code>config.py</code> no se encuentra	Ejecutar los notebooks teniendo <code>deepgravity/preprocessing/</code> como directorio de trabajo.

17. Cierre

Deep Gravity Chile deja implementada una cadena completa para transformar productos telco y territoriales en los formatos requeridos por Deep Gravity, entrenar el modelo y auditar sus predicciones.

La principal utilidad del repositorio es proporcionar una implementación de referencia documentada y extensible, aplicable a otras fuentes de estructura equivalente. Su reutilización debe mantener la distinción entre ejecutabilidad computacional, desempeño exploratorio y validez sustantiva de los flujos analizados, así como las restricciones de uso que rigen los productos derivados de fuentes de telefonía móvil.

La atribución y forma de citación del código original deben mantenerse conforme a los términos de la licencia CC BY 4.0 bajo la cual sus autores lo publicaron, y a los archivos que esta entrega conserva sin modificación:

- `README_original_DeepGravity.md`
- `CITATION.cff`

La Sección 2 detalla la licencia aplicable, las condiciones que impone y la forma en que esta entrega las cumple.